TIPS ACADEMY

Academy Management System

Product Requirements Document | v2.0 | March 2026

Status	Approved for Development
Version	v2.0 - Updated (Photo, Storage, Installer, Performance)
Platform	Electron + Next.js Desktop App (Windows)
Database	SQLite (local, offline-first)
Backend	Node.js + Express (embedded in Electron)

SECTION 1 | EXECUTIVE SUMMARY

1. Executive Summary

Tips Academy Management System (TAMS) is a fully offline-capable desktop application that centralizes all administrative, academic, and communication operations of Tips Academy. Delivered as a professional Windows installer, the client receives a single setup binary with zero server or runtime dependency. The entire application — database, student photos, PDF receipts, and backups — lives in one organized folder on the client machine, making it simple to locate, back up, and migrate.

1.1 Goals

- Eliminate manual paperwork for admissions, fee tracking, and attendance
- Provide real-time per-student profiles with photo, academic records, and financial history
- Automate parent communication via WhatsApp broadcast and individual alerts
- Generate printable PDF receipts, report cards, and progress reports
- Support multiple batches with sub-groups (e.g., Biology / Computer) sharing the same classroom
- Deliver a professional Windows installer — shortcuts, auto-start, and system tray all configured automatically

1.2 Tech Stack

Layer	Technology
Frontend	Next.js 14 (App Router) — compiled to static export, served locally inside Electron
Desktop Shell	Electron — packages the app as a Windows NSIS .exe installer via electron-builder
Backend / API	Node.js + Express — embedded local server, spawned by Electron main process
Database	SQLite via better-sqlite3 — synchronous, zero-config, single .db file
PDF Generation	Puppeteer (headless Chrome) for report cards; pdfkit for lightweight receipts
WhatsApp	Meta Cloud API (primary) + whatsapp-web.js (fallback if Meta not verified)
Photo Storage	Local filesystem under data root — photos auto-resized and stored as JPEG
Installer	electron-builder + NSIS — one-click Windows installer with shortcuts and auto-start

SECTION 2 | DATA STORAGE & FILE ORGANISATION

2. Data Storage & File Organisation

All application data lives in a single root folder. This makes it trivially easy for the academy to back up everything by copying one folder to a USB drive or cloud storage. Nothing is scattered across multiple locations.

2.1 Root Data Directory

Default Location
C:\Users\[Username]\AppData\Local\TipsAcademy\ This path is displayed in Settings and can be changed to any location (e.g., a shared network drive) during first-run setup.

2.2 Folder Structure

Path (relative to data root)	Contents
tams.db	The SQLite database — all student records, fees, attendance, exams, logs
students/photos/	Student profile photos. Filename: {student_id}_{timestamp}.jpg
pdfs/receipts/	Generated fee receipt PDFs. Filename: receipt_{receipt_no}_{date}.pdf
pdfs/reportcards/	Monthly report card PDFs. Filename: {student_id}_{month}_{year}.pdf
pdfs/attendance/	Exported attendance report PDFs
backups/	Daily auto-backups of tams.db. Format: tams_backup_{YYYY-MM-DD}.db
logs/	Application error logs for debugging — auto-rotated, max 30 days kept

2.3 Student Photo Handling

- Staff click 'Upload Photo' on the Add/Edit Student form and select any JPG or PNG
- File is copied into students/photos/ and renamed to {student_id}_{timestamp}.jpg
- The database stores only the relative file path — not the image binary itself
- Photos display as thumbnails in the student list, in the Student Profile header, on attendance roll call, and on printed report cards
- If no photo is set, a default avatar placeholder is shown everywhere in the UI
- Updating a photo replaces the disk file and updates the path; the old file is deleted
- Supported formats: JPG, JPEG, PNG. Auto-resized to 400x400px on import to control disk usage

2.4 Backup & Restore

- Automatic daily backup: on app startup, if today's backup is missing, tams.db is copied to backups/
- Manual backup: 'Back Up Now' button in Settings creates an immediate timestamped backup
- Restore: staff select any backup file in Settings; app restarts automatically after restore
- Retention: backups older than 60 days are auto-deleted to save disk space (configurable)

> **Moving to a New PC**
> Copy the entire TipsAcademy data folder to the new machine. Install TAMS on the new machine. On first run, point the app to the copied folder. All data, photos, receipts, and history will be fully intact.

SECTION 3 | SYSTEM ARCHITECTURE

3. System Architecture

3.1 How It Works

Electron's main process spawns the Express server and communicates the dynamically assigned local port to the renderer via contextBridge IPC. Next.js is pre-compiled to a static export at build time and loaded from disk. SQLite queries are synchronous (better-sqlite3), so there are no async delays on database reads. The renderer never touches the filesystem directly — all file operations (photos, PDFs) go through the Express API.

3.2 Monorepo Folder Layout

Path	Purpose
/electron	Main process, IPC bridge, window management, tray icon, auto-updater
/renderer (Next.js)	All UI pages, components, hooks, and client-side state
/server (Express)	REST API route handlers and controllers
/server/db	SQLite schema, Knex migrations, seed data
/server/services	FeeService, WhatsAppService, PDFService, AttendanceService, StorageService
/shared	TypeScript types and constants shared by renderer and server

3.3 WhatsApp Option Comparison

Criterion	Meta Cloud API (Primary)	whatsapp-web.js (Fallback)
Type	Official Meta Business API	Unofficial browser automation
Cost	Free up to 1,000 conversations/month	Free, open-source
Reliability	High — official support and SLA	Medium — breaks on WhatsApp app updates
Setup	Meta Business account + verified number	Scan QR code — instant start
PDF Attachments	Yes — send as document messages	Yes
Recommendation	Use for production delivery	Ship as fallback / use in development

SECTION 4 | DATABASE SCHEMA

4. Database Schema (SQLite + Knex Migrations)

All data lives in one .db file. Knex.js manages schema migrations so upgrades are applied cleanly without data loss. Indexes are created on all foreign keys and frequently filtered columns for fast queries.

Table	Key Columns	Purpose
batches	id, name, grade, year, is_active	Class-year groups — root organizer for all data
groups	id, batch_id, name	Sub-streams within a batch (Biology, Computer, etc.)
students	id, name, gender, dob, batch_id, group_id, photo_path, status, admission_date	Student master — photo_path is relative path in data folder
student_contacts	id, student_id, parent_name, phone, whatsapp_number, relation	Parent / guardian contacts
fee_structures	id, batch_id, monthly_amount, discount_default, admission_fee	Default fee rules per batch
student_fees	id, student_id, month_year, amount_due, discount, amount_paid, paid_date, receipt_no, status	Monthly fee ledger
attendance	id, student_id, batch_id, date, status (P/A/L), marked_by	Daily attendance records
exams	id, batch_id, group_id, subject, exam_type, date, max_marks	Exam / quiz / test definitions
exam_results	id, exam_id, student_id, marks_obtained, remarks	Per-student exam scores
whatsapp_logs	id, student_id, parent_number, message_type, status, sent_at	Full WhatsApp message audit trail
settings	key, value	App config: fees, API keys, data folder path, branding

SECTION 5 | MODULE SPECIFICATIONS

5. Module Specifications

5.1 Campus — Batches & Groups

Batches are the root organizer. Every student, exam, attendance record, and fee entry belongs to a batch. Groups within a batch handle sub-streams where students share a classroom but have separate exam tracking.

Batch Features

- Create with: Grade (9 / 10 / 11 / 12 / Custom), Year, Display Name, Default Monthly Fee, Default Admission Fee
- Archive old batches without deleting historical data
- Batch dashboard: total students, attendance summary, fee collection summary, upcoming exams

Group Features

- Create named groups inside any batch (Biology Group, Computer Group, or any custom name)
- Each student is assigned to exactly one group
- Groups filter exam results, attendance reports, and WhatsApp messages by sub-stream

5.2 Student Management

Student List View

- Table with student photo thumbnail, name, batch, group, status — filters by Batch, Group, Status, Gender
- Search by name, phone number, or student ID
- Click any row to open full Student Profile

Add / Edit Student Form (includes Photo Upload)

Field	Required	Notes
Student Photo	No	Click to browse or drag-and-drop JPG/PNG. Previewed inline. Auto-resized to 400x400px. Saved to students/photos/ in data folder.
Full Name	Yes	
Gender	Yes	Male / Female / Other
Date of Birth	No	

Field	Required	Notes
Batch	Yes	Dropdown from active batches
Group	Yes	Auto-filtered by selected batch
Parent Name	Yes	
Parent Phone	Yes	Standard call number
Parent WhatsApp Number	Yes	May differ from phone; used for all automated WhatsApp messages
Admission Fee	Yes	Pre-filled from batch default; editable; can be zero for full waiver
Monthly Fee Discount	No	Fixed PKR amount or % — auto-applied to every monthly fee record
Admission Date	Yes	Defaults to today
Address / Notes	No	Free text field

Student Profile Page

Clicking any student opens a full-page tabbed profile with all linked records:

- Overview: student photo prominently displayed, personal info, contacts, batch/group, admission date
- Attendance: monthly calendar heatmap — P (green) / A (red) / L (yellow); cumulative counts and percentage
- Fees: month-by-month ledger with status chips; outstanding balance summary; receipt download links
- Exams: all quiz and test results with subject, date, marks obtained, batch rank
- WhatsApp: all messages sent to this parent — type, timestamp, delivery status

5.3 Attendance

Daily Attendance Marking

- Select Batch and Date (defaults to today) — student list loads instantly
- Each row shows student photo thumbnail, name, and P / A / L radio toggle for fast visual confirmation
- Bulk-mark all Present then flip individual absentees — attendance for 40 students done in under 60 seconds
- Absent students automatically queued for WhatsApp absence alert after save

Attendance Reports

- Monthly summary grid: students as rows, days as columns, color-coded cells
- Export to PDF for a batch or individual student; saved to pdfs/attendance/
- Attendance percentage shown prominently on every Student Profile

5.4 Fee Management

Monthly Fee Generation

- Auto-runs on 1st of month (or triggered manually): creates Pending record for every active student
- Amount Due = Batch Monthly Fee minus the student's individual discount (flat or %)
- Prior-month outstanding balances surfaced alongside current dues

Fee Collection Entry

- Select student (photo shown for visual confirmation) — due amount pre-filled
- Enter Amount Paid, Payment Date, Payment Mode (Cash / Bank Transfer / Other)
- Status auto-updates to Pending, Partially Paid, or Fully Paid

Fee Receipt (PDF)

- Generated after every payment: academy logo, receipt number, student name and photo, batch, month, amounts
- Saved to pdfs/receipts/ — printable directly or sent as WhatsApp attachment from same screen

5.5 Examination Management

Exam Types

Type	Description
Random Quiz	Teacher-initiated any time; single subject; ad hoc
Weekly Test	End-of-week; one subject per day across several days
Board Prep Test	Daily intensive tests during pre-board season alongside regular classes
Monthly Report	Auto-aggregated from all quizzes and weekly tests — no separate entry

Marks Entry & Report Cards

- Marks entered in an inline grade sheet — student photo visible for identification
- Absent students receive an 'Absent' marker, not zero
- Monthly report card PDF: student photo, subject-wise table, attendance summary, teacher remarks
- Report cards saved to pdfs/reportcards/ and can be sent directly to parent WhatsApp from the UI

5.6 WhatsApp Communication

Automated Monthly Fee Alerts

Trigger	Date(s)	Action
Broadcast 1	1st of month	Friendly reminder to ALL parents with pending dues: month, amount, pay-by date
Broadcast 2	5th of month	Second reminder personalized with student name and exact amount still outstanding
Individual Alert	8th to 11th daily	Targeted message to each parent with unpaid balance — staff review queue before sending

Absence Alerts, Manual Sends & Message Log

- Absent students trigger a parent WhatsApp alert; staff can suppress from queue before send
- Compose and send custom messages to individual parents, batch, or all parents academy-wide
- Attach any PDF (receipt, report card) as a WhatsApp document message
- Full log: student photo, name, number, message type, timestamp, status (Sent / Delivered / Failed); one-click retry

SECTION 6 | PROFESSIONAL INSTALLER & PERFORMANCE

6. Professional Installation & Performance

6.1 Installer Design (electron-builder + NSIS)

The application ships as a single Windows installer. Built with NSIS via electron-builder, the experience mirrors professional software like VS Code or Zoom. The client runs one .exe once — they never need to run it again.

Installation Flow

Step	Description
1	Client double-clicks TipsAcademySetup.exe — branded wizard opens with academy logo and name
2	Install location choice (default: C:\Program Files\TipsAcademy)
3	Options: 'Create Desktop shortcut' (checked), 'Pin to Taskbar' (checked), 'Launch at Windows login' (checked)
4	Progress bar — installation completes in under 30 seconds on any modern machine
5	Completion screen: 'Launch Tips Academy now' checkbox — app starts immediately
6	First-launch setup wizard: academy name, logo, default fees, data folder location
7	App appears in Windows 'Add or Remove Programs' for clean uninstall at any time

Auto-Start & System Tray

- When 'Launch at Windows login' is enabled, a Windows Registry entry launches the app minimized to the system tray on every login
- System tray icon: right-click shows 'Open TAMS', 'Check WhatsApp Queue', 'Exit'
- Staff can toggle auto-start at any time in Settings
- Client never needs to find or run the .exe again after the initial install

Updates

- electron-updater checks for new versions silently at startup
- Updates download in the background; user sees: 'Update ready — restart to apply'
- Never forces a restart mid-session

> **What the Client Receives**
> One file: TipsAcademySetup_v1.0.exe (approx. 80-120 MB). Run it once, follow the 5-step wizard — shortcuts are created, auto-start is configured. Done. The app is in the Start Menu, Desktop, Taskbar, and system tray.

6.2 Performance Engineering

Performance is a first-class requirement. Every architectural choice prioritizes speed and zero lag.

Database

- better-sqlite3: synchronous driver — queries return in microseconds with no async overhead
- Indexes on all foreign keys and filtered columns (student_id, batch_id, date, month_year)
- Settings, active batches, and student list cached in memory at startup — no repeated disk reads
- Write-Ahead Logging (WAL) mode for faster concurrent reads without blocking writes
- All writes wrapped in transactions — no partial writes on crash or power loss

Frontend

- Next.js static export: pages pre-rendered to HTML at build time — zero server round-trip to display a page
- React Query: data cached in memory; background re-fetch only when stale — navigating back to a page is instant
- Student photos loaded lazily — only visible viewport images are fetched (Intersection Observer)
- Virtualized tables via react-virtual — 500-student list renders in the same time as a 10-student list
- PDF export runs in Electron main process — the UI never freezes during generation

Performance Targets

Operation	Target
App cold start to usable UI	< 3 seconds
Student list (500 students)	< 200ms
Student profile open	< 150ms
Sidebar page navigation	< 80ms
Attendance batch save	< 100ms
Fee record + status update	< 50ms
Fee receipt PDF generation	< 2 seconds
Monthly report card PDF	< 4 seconds
WhatsApp single message send	< 3 seconds

Reliability & Error Prevention

- Input validated on both frontend and backend — malformed data never reaches SQLite
- WhatsApp sends: 3 auto-retries with exponential back-off before marking Failed
- All exceptions logged to logs/ — user sees a friendly dialog, never a stack trace
- Destructive actions require typed confirmation — no accidental data loss

SECTION 7 | NAVIGATION & UI

7. Sidebar Navigation Structure

Sidebar Item	Description
Dashboard	Academy overview: today's attendance, fee collection today, pending alerts, upcoming exams
Campus > Batches	All batches list; create or edit a batch; open batch dashboard
Campus > Groups	Manage sub-groups within each batch
Students	All students with photo thumbnails and filters; add new student; click row for profile
Attendance	Mark daily attendance by batch; view monthly report; export to PDF
Fees	Fee collection entry; monthly status board; outstanding dues list; receipt viewer
Exams	Exam list; create exam; enter marks; generate and send report cards
WhatsApp	Message queue; broadcast composer; absence alert queue; message log; template editor
Settings	Academy profile, fee defaults, WhatsApp API config, data folder path, backup and restore

UI Principles: Dark blue sidebar (#1E3A5F) with white text. Card-based content area. Destructive actions require confirmation dialogs. Toast notifications for async operations. Keyboard shortcuts: Ctrl+N (new record), Ctrl+S (save), Ctrl+P (print/PDF). Fully functional offline; internet only needed for WhatsApp sends.

SECTION 8 | BACKEND API ROUTES

8. Backend API Design (/api/v1)

Express runs on a dynamic localhost port communicated to the renderer via Electron IPC. All routes return JSON. Photo uploads use multipart/form-data. PDF-generating routes return a local file path for the renderer to open or send.

Method	Route	Description
GET	/batches	List all batches
POST	/batches	Create a batch
GET/ POST	/batches/:id/groups	List or create groups within a batch
GET	/students	List students with filters (batch_id, group_id, status)
POST	/students	Create student; multipart/form-data handles photo upload
PUT	/students/:id	Update student; handles photo file replacement
GET	/students/:id/profile	Full student profile — all linked records in one response
GET	/students/:id/photo	Serve student photo file from disk
GET	/attendance	Get attendance for a batch on a given date
POST	/attendance/bulk	Save entire batch attendance in one atomic write
POST	/fees/generate	Generate monthly fee records for all active students
PATCH	/fees/:id/pay	Record a payment; status auto-updates
GET	/fees/:id/receipt	Generate and save PDF receipt; returns file path
POST	/exams	Create an exam definition
POST	/exams/:id/results	Bulk save marks for all students in the exam
GET	/exams/report/:studentId/:month	Generate report card PDF; returns file path
POST	/whatsapp/broadcast	Send broadcast to batch or all parents
POST	/whatsapp/send	Send individual message or PDF attachment to one number
GET	/whatsapp/queue	List pending messages awaiting staff review
GET	/whatsapp/logs	Full audit log with filters
POST	/storage/backup	Trigger manual backup to backups/ folder
GET	/storage/backups	List available backup files with dates and sizes
POST	/storage/restore	Restore from selected backup; app restarts after

Method	Route	Description
GET/PUT	/settings/:key	Read or update a settings value

SECTION 9 | DEVELOPMENT PHASES

9. Development Plan

Phase	Timeline	Deliverables
1 — Foundation	Weeks 1-2	Monorepo scaffold; Electron + Next.js + Express wired via IPC; SQLite schema and Knex migrations; data folder creation; Batch and Group CRUD; Student CRUD with photo upload and storage
2 — Core Modules	Weeks 3-5	Attendance marking with photo roll call; fee generation, collection, and receipt PDF; exam creation, marks entry, and report card PDF; Student Profile all tabs
3 — Profiles & Dashboard	Week 6	Full Student Profile (5 tabs with photos throughout); Academy Dashboard with live stats; all PDFs saved to correct subfolders
4 — WhatsApp	Weeks 7-8	Meta Cloud API integration; 1st and 5th broadcast scheduler; 8th-11th individual alerts with review queue; absence alert auto-queue; PDF attachment sending; message log and retry
5 — Packaging & UAT	Week 9	electron-builder NSIS installer; desktop shortcut, taskbar pin, auto-start prompts; system tray icon; electron-updater; backup/restore; performance tuning; client UAT; final delivery

SECTION 10 | NON-FUNCTIONAL REQUIREMENTS

10. Non-Functional Requirements

Requirement	Specification
Performance	All local DB queries < 300ms. PDFs < 4s. Page nav < 80ms. App cold start < 3s.
Offline First	100% functional without internet. WhatsApp messages queue locally and retry on reconnect.
Data Storage	All data in one folder (DB + photos + PDFs + backups). Path configurable in Settings.
Data Safety	All DB writes transactional. Daily auto-backup. Manual backup in Settings.
Scalability	2,000+ students and 5+ years of data without degradation.
Installer	One .exe. Desktop shortcut, Taskbar pin, Start Menu — all offered in wizard.
Auto-Start	Optional launch at Windows login, minimized to system tray. Toggleable in Settings.
Updates	Silent background download. Restart prompt to apply. Never forced mid-session.
Error Handling	No silent crashes. All errors logged. Users see friendly messages, not stack traces.
Usability	All core functions operable by staff after a single training session.

11. Open Questions

Question	Needed By
Will the client complete Meta Business verification, or use whatsapp-web.js initially?	Before Phase 4
Should fee broadcasts auto-send or always require staff approval from the queue?	Phase 3 review
Is multi-user / multi-PC access needed (affects architecture significantly)?	Phase 1 kickoff
Are subjects per batch configurable, or a shared fixed list across all batches?	Before exam build
Should cloud sync or remote access be planned for v2.0?	Post v1 roadmap

End of Document | Tips Academy Management System | PRD v2.0 | March 2026